

INTEGRANTES:

Juan Pablo Cuervo Contreras

Carnet: 1000100656

David Felipe Ortiz Salcedo

Carnet: 10000100448

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Herencias e Interfaces.

2026-1 — Laboratorio 3

OBJETIVOS

Desarrollar competencias básicas para:

1. Aprovechar los mecanismos de la herencia y de interfaces.
2. Organizar las fuentes en paquetes.
3. Usar la utilidad `jar` de java para entregar una aplicación.
4. Extender una aplicación cumpliendo especificaciones de diseño, estándares y verificando su corrección.
5. Vivenciar las prácticas XP :
Codign. Code must be written to agreed [standards](#).
Testing. Code the [unit test](#) first.
6. Utilizar los programas básicos de java (`javac`, `java`, `javadoc`, `jar`), desde la consola.

ENTREGA

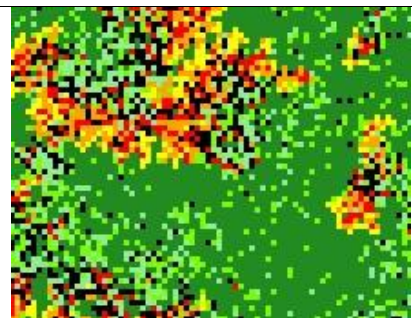
- Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (Casallas-Duarte) Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Desarrollar un simulador para **Forest Fire Model**

El **Forest Fire Model** fue introducido como un autómata celular para estudiar auto-organización y criticidad en sistemas dinámicos. [\[SIMULADOR\]](#)

Nuestro sistema Forest Fire será un bosque en el que se encuentran árboles, fuego y fuentes de agua. Todos representados en una cuadrícula bidimensional. El bosque se modifica en una secuencia de pasos discretos (tics). En cada paso cada agente (árbol, fuego, agua) actúa en orden de izquierda a derecha y de norte a sur.



PARTE I. Conociendo el proyecto

A Revisando línea base [En [lab03.doc](#)]

1. En el directorio descarguen los archivos contenidos de [forest.zip](#). Revisen el código:
 - (a) ¿Cuántos paquetes tiene?
R/: Tiene 3 paquetes:
 - El paquete principal

- El paquete de presentación
- El paquete de dominio

(b) ¿Cuál es el propósito del paquete presentación?

R/: El propósito del paquete de presentación es mostrar cómo se ve visualmente el simulador Forest Fire Model.

(c) ¿Cuál es el propósito del paquete dominio?

R/: El propósito del paquete de dominio es ser la fachada.

2. Revisen el paquete de dominio,

(a) ¿Cuáles son los diferentes tipos de componentes de este paquete?

R/:

- Clase abstracta: La clase LivingThing.
- Interfaz: La interfaz Thing.
- Clase Concreta: La clase Forest y la clase Tree.

(b) ¿Qué implica cada uno de estos tipos de componentes?

R/:

- Clase abstracta: implica que los métodos y atributos los puede heredar sus clases hijas (en este caso, LivingThing es la clase padre y Tree es la clase hija).
- Interfaz: agrupa clases que pueden tener una relación y pueden cambiar a la vez (Forest y LivingThing).
- Clase Concreta: La plantilla que contiene atributos y métodos (clase Forest y clase Tree).

3. Revisen el paquete de presentación,

(a) ¿Cuántos componentes tiene?

R/: Sólo tiene un componente que es la clase ForestGUI.

(b) ¿Cuántos métodos públicos propios (no heredados) ofrece?

R/: Ofrece 2 métodos públicos propios:

- gettheForest()
- main

4. Para ejecutar un programa en java,

(a) ¿Qué método se debe ejecutar?

R/: El método main se usa para ejecutar un programa en java.

(b) ¿En qué clase se encuentra?

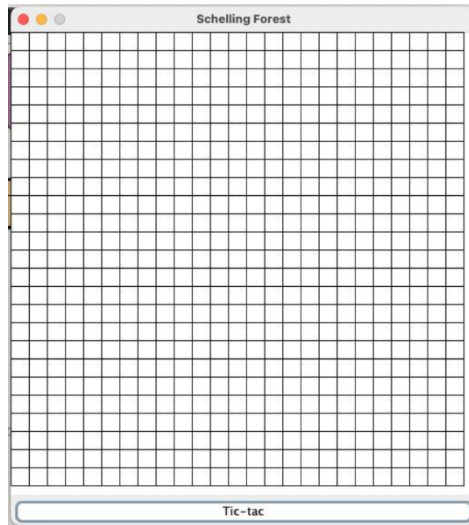
R/: Se encuentra en la clase ForestGUI.

Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo

5. Ejecuten el programa.

(a) ¿Qué funcionalidades ofrece?

R/:



(b) ¿Qué hace actualmente?

R/: De momento, sólo crea el tablero con una rejilla cuadriculada llamada Schelling Forest y un botón con nombre “Tic-tac”.

(c) ¿Por qué?

R/: Porque es el único método que está implementado.

B Realizando ingeniería inversa. Arquitectura General [En [lab03.doc](#) [forest.astah](#)]

1. Consulten el significado de las palabras [package](#) e [import](#) de java.

(a) ¿Qué es un paquete?

R/: Es un conjunto lógico de clases e interfaces relacionadas que se agrupan bajo un mismo nombre para organizar mejor el código y evitar conflictos de nombres.

(b) ¿Para qué sirve?

R/: Sirve para hacer más fácil la modularidad del código (dividir el programa en partes más pequeñas y funcionales).

(c) ¿Para qué se importa?

R/: Para hacer referencia a tipos definidos en otros paquetes.

(d) Expliquen su uso en este programa.

R/: El uso de la interfaz Tree es para agrupar la lógica de las clases LivingThing, Tree, Forest.

2. Revisen el contenido del directorio de trabajo y sus subdirectorios. Describan su contenido.

(a) ¿Qué coincidencia hay entre paquetes y directorios?

R/: La relación es que los paquetes son espacios de nombres lógicos en Java y el sistema de archivos los representa mediante una estructura de directorios que refleja el nombre del paquete (por ejemplo, `com.miempresa.app` → `com/miempresa/app`).

3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle. **En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram)**

R/:



C Realizando ingeniería inversa. Arquitectura detallada [En lab03.doc forest.astah]

1. Para preparar el proyecto para MDD. Completen el diseño detallado del paquete de dominio.
 - (a) ¿Qué componentes hacían falta?
R/: Hacían falta los componentes Thing (Interfaz) y Tree (Clase Concreta). De igual forma, dejar LivingThing (Clase Abstracta) y Forest (Clase Concreta) en el paquete, ya que estas clases existían en el paquete de presentation.
2. Completen el diseño detallado del paquete de presentación.
 - (a) ¿Por qué hay dos clases y un archivo .java?
R/: Porque hay una clase ForestGUI que tiene una clase interna llamada PhotoForest. Así mismo, .java arma el paquete.
3. Adicione la clase de pruebas necesaria para BDD en un paquete independiente de test. (No lo adicione al diagrama de clases)
 - (a) ¿Qué clase debe importar?
R/: Debe importar las clases del paquete de dominio.
 - (b) ¿Por qué?
R/: Porque estas clases contienen la lógica de Forest Fire Model, es decir, se realizan pruebas en estas clases: Thing, Tree, LivingThing y Forest con el fin de validar que se cumplen con los requerimientos y el diseño del simulador.

PARTE II. Desarrollando el proyecto

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

A Ciclo 1. Iniciando con árboles [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

1. Estudie la clase Forest.
 - (a) ¿Qué tipo de colección usa para albergar cosas?
R/: Usa la colección array de arrays (atributo places) para guardar el lugar de la cosa.
 - (b) ¿Puede tener árboles?
R/: Sí puede tener árboles.
 - (c) ¿Por qué?
R/: Porque el método setThing, se encarga de establecer qué es la cosa, además de que puede tener como parámetro Thing.
2. Estudie el código asociado a la clase Tree,

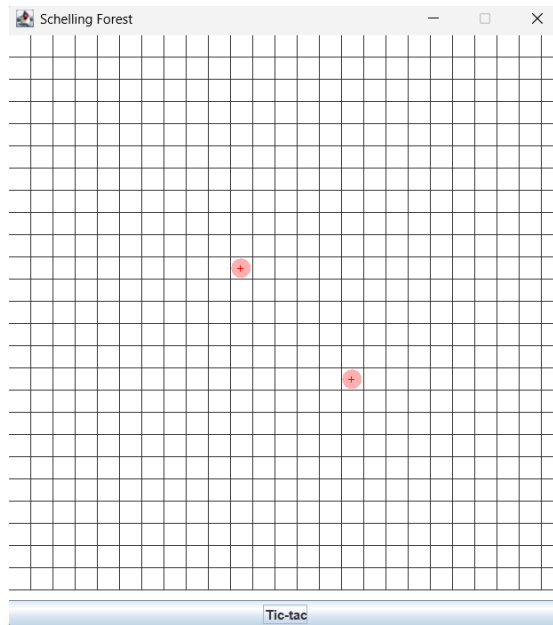
- (a) ¿de qué color es?
R/: El color que tiene por defecto es pink.
- (b) ¿qué forma usa para pintarse?
R/: Utiliza la forma ROUND.
- (c) ¿cuándo cambia de color?
R/: Cambia de color dependiendo de si los pasos (tictacs) al dividir por 4, el residuo es divisor de 4 o sobra 1 o 2 en la división.
- (d) ¿qué componentes definen la clase **Tree**? Justifique sus respuestas.
R/: Los componentes LivingThing (Clase Abstracta) y Thing (Interfaz).
3. **Tree** por ser un **LivingThing**,
- (a) ¿qué atributos tiene?
R/: Tiene el atributo years y energy, ya que LivingThing al ser una clase abstracta, no necesita volver a declararla de nuevo. Así mismo, el atributo years su visibilidad es protected, por lo que puede ser usada en todo el paquete domain. Finalmente, Tree hereda el atributo energy, ya que al no modificarla directamente, puede interactuar con ella.
- (b) ¿qué puede hacer (métodos)?
R/: Puede tener los métodos step, getEnergy, isLivingThing.
- (c) ¿qué decide hacer distinto (sobre-escritura)?
R/: Nada.
- (d) ¿qué no puede hacer distinto (métodos finales)?
R/: No puede cambiar los métodos step, getEnergy e isLivingThing. Dado que la clase abstracta LivingThing permite heredar estos métodos, la clase hija Tree no puede cambiar el cómo se pueden comportar.
- (e) ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.
R/: Debe aprender a hacer el método step, que permite determinar el comportamiento de una cosa basada en su energía.
4. Por comportarse como un **Thing**,
- (a) ¿qué sabe hacer?
R/: Sabe hacer los métodos tictac, shape, getColor, isOnlyThing, isLivingThing.
- (b) ¿qué decide hacer distinto?
R/: Decide tener un color distinto (color) con el método getColor y una forma distinta (ROUND) con el método shape. Así mismo, se comporta de manera distinta con el método ticTac donde si los pasos (tictacs) al dividir por 4, el residuo es divisor de 4 o sobra 1 o 2 en la división, sus años aumentan o puede morir.
- (c) ¿qué no puede hacer distinto?
R/: Dado que LivingThing también hereda el método isLivingThing de Thing pero esta clase abstracta lo tiene definido como final, entonces la clase Tree no puede cambiar su comportamiento.
- (d) ¿qué debe aprender a hacer? Justifique sus respuestas.
R/: Debe aprender a tomar el método isOnlyThing, que por lo que deducimos, es verificar si es una sola cosa o no.
5. De acuerdo con lo anterior un **Tree**,
- (a) ¿Cómo actúa?
R/: Un Tree es un árbol que empieza siendo rosado, con una forma redonda y que actúa de acuerdo con los pasos que da, que por herencia de LivingThing, puede reducir su energía tornándose verde o naranja luego de varios años.
6. Ahora vamos a crear dos árboles en diferentes posiciones (10,10) (15,15), llámelos **beard** y **soul**, usando el método **someThings**. Ejecuten el programa,
- (a) ¿Qué pasa con los árboles?
R/: No cambian.

(b) ¿Por qué?

R/: Porque aún no está implementado el método que les permite cambiar de color a través del tiempo (ticTac).

(c) Capturen una pantalla significativa.

R/:

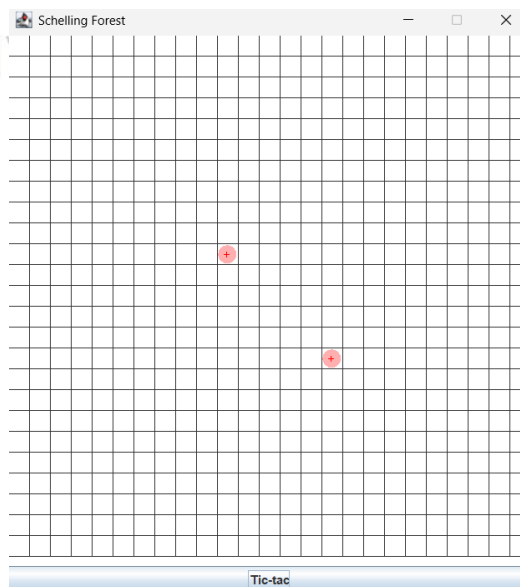


7. Diseñen, construyan y prueben el método llamado `ticTac()` de la clase `Forest`.

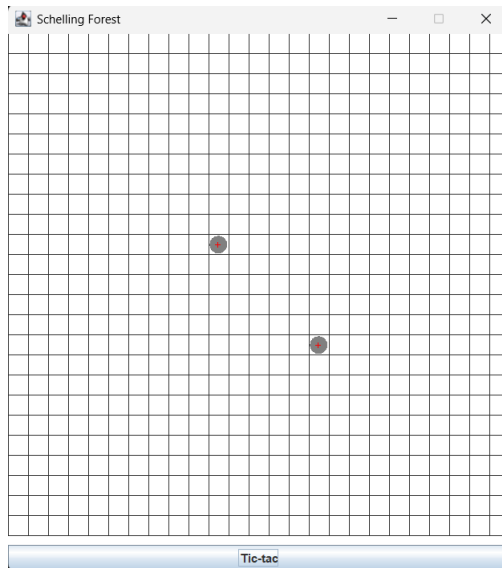
8. (a) ¿Cómo quedarían `beard` y `soul` después de 4, 7, 10 y 12 tic-tac? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes.

R/:

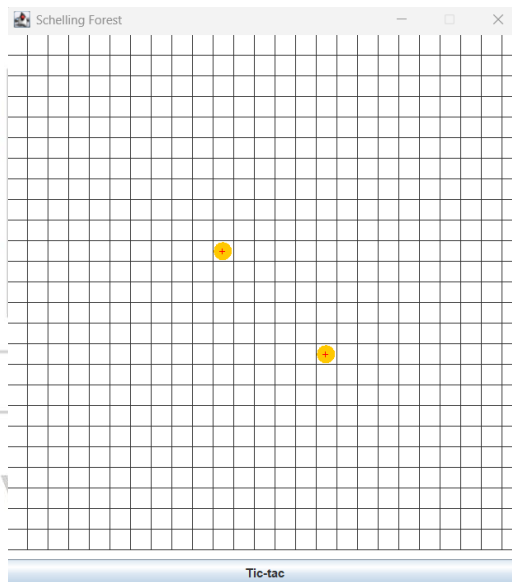
- Después de 4 tic-tac:



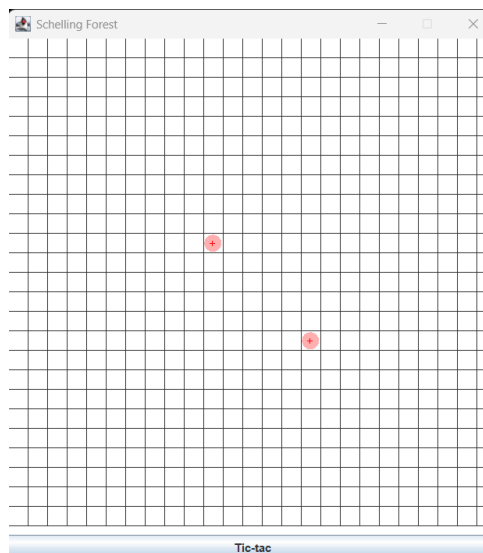
- Después de 7 tic-tac:



- Después de 10 tic-tac:

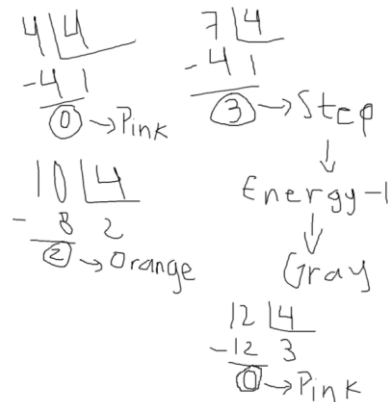


- Después de 12 tic-tac:



(b) ¿Es correcto?

R/: Sí es correcto, de acuerdo con los cálculos hechos en la siguiente gráfica:



Analizando cada momento, se aprecia que en 4 tic tacs, los árboles son de color rosado dado que el residuo obtenido es 0. En 7 tic tacs, el residuo obtenido es 3 por lo que la energía de los árboles disminuye y tienen más años, denotados con el color gris. Por consiguiente, en 10 tic tacs, los árboles se tornan de color naranja dado que el residuo obtenido es 2 y finalmente, en 12 tic tacs, el residuo es 0 por lo que los árboles son de color rosado. Los árboles mueren en 203 tic tacs.

B Ciclo 2. Incluyendo ardillas [En `lab03.doc forest.astah *.java`]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir tener ardillas en el bosque. Ellas (i) son cafés y a medida que pasan los años se van tornando amarillentas, (ii) después de 10 años mueren, (iii) si quedan dos vecinas con una celda intermedia vacía, allí nace una nueva ardilla, (iv) se mueven aleatoriamente por el bosque

1. Para implementar esta nueva [LivingThing](#)

(a) ¿cuáles métodos se sobre-escriben (overriding)?

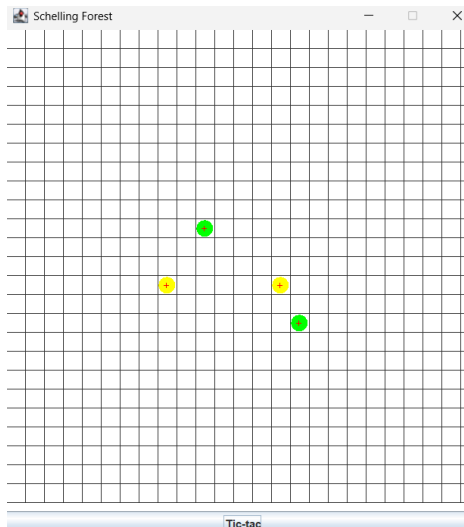
R/: Se sobrescribe el método `step`, ya que la ardilla se comporta de manera diferente a medida que pasan los años.

2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad).

3. Adicione una pareja de ardillas, de nombre `scrat` y `sandy`,

(a) ¿Cómo quedarían después de 5 Tic-tac? Ejecuten el programa y hagan 5 clics en el botón. Capturen una pantalla significativa.

R/: Quedarían de color amarillo de acuerdo con la lógica de que van cambiando de color a medida que pasan los años (en este caso pasaron 3 años).



(b) ¿Es correcto?

R/: Si, es correcto, ya que implementamos la lógica de que cada 3 años, la ardilla cambia de color.

C Ciclo 3. Adicionando sombras [En `lab03.doc forest.ajah *.java`]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es incluir sombras (sólo vamos a permitir el tipo básico). Ellas (i) son puntos negros que esparcen su sombra en toda la fila en la que se encuentran, (ii) se desplazan de sur a norte circularmente

1. Para poder adicionar sombras,

(a) ¿debe cambiar en el código de `Forest` en algo?

R/: No debe cambiar el código.

(b) ¿por qué?

R/: Porque la clase `Forest` ya sabe manejar objetos tipo `Thing` y llamar al método `ticTac`.

(c) ¿debe extender `Thing`?

R/: Si debe implementar la Interfaz `Thing`.

(d) ¿por qué?

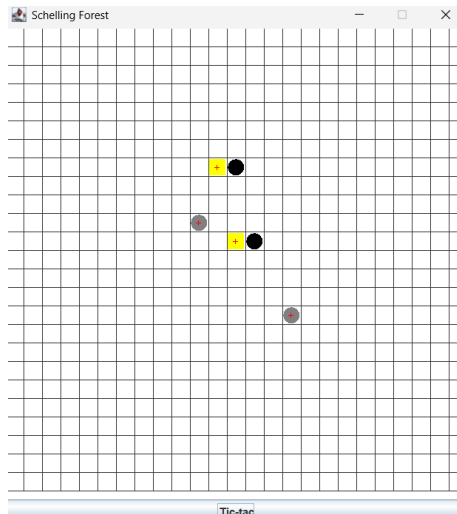
R/: Porque debe cumplir con el contrato de la Interfaz, además de que la clase `forest` pueda poner las cosas dentro de `places` (matriz).

2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad).

3. Adicionen dos objetos sombras en `Forest`, llámenlos `thief` y `lass`,

(a) ¿Cómo quedarían después de cuatro Tic-tac? Ejecuten el programa y hagan 7 clics en el botón. Capturen una pantalla significativa.

R/:



(b) ¿Es correcto?

R/: Si es correcto, ya que la sombra sigue a la cosa que pertenece esta (en este caso a la ardilla).

D Ciclo 4. Proponiendo y diseñando: Nuevo árbol [En `lab03.doc forest.astah *.java`]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir en un nuevo tipo de `Tree`.

1. Propongan, describan e implementen el nuevo tipo de `Tree`. (Mínimo dos pruebas de unidad)
2. Considerando una pareja de ellos con el apellido de ustedes.

(a) Piensen en otra prueba significativa y expliquen la intención.

(b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.

(c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

R/:

Implementación: Se creó la clase `Pine` que extiende de `Tree`.

Diferencias clave:

- **Energía:** El pino es más resistente, iniciando con una energía de 150 en comparación con la energía base de 100 de `LivingThing`.
- **Apariencia:** A diferencia del `Tree` normal que cambia de color según la estación (`season`), el `Pine` sobrescribe el método `ticTac()` para mantener siempre un color verde oscuro (`Color(0, 100, 0)`).
- **Ubicación:** Se instanció un pino en la posición (8, 8) dentro del método `someThings()` de la clase `Forest`.

E Ciclo 5. Proponiendo y diseñando: Nueva cosa [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir una nueva clase de **Thing** (no **Tree**) en el **Forest**

1. Propongan, describan e implementen la nueva clase de **Thing**. (Mínimo dos pruebas de unidad)
2. Considerando un par de ellos con el nombre de ustedes.

(a) Piensen en otra prueba significativa y expliquen la intención.

(b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.

(c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

Propósito: Introducir un agente que mitigue los incendios.

Comportamiento: El Firefighter busca la celda con fuego más cercana utilizando la distancia Manhattan. Si encuentra fuego, se desplaza hacia él. Al llegar o moverse, deja Earth (Tierra) en su posición anterior para evitar celdas vacías. Implementa la interfaz Thing y hereda de LivingThing, lo que le permite tener un ciclo de vida y ser representado en el bosque con una forma circular (ROUND) de color azul.

PARTE III. Desarrollando el reto [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

En la zona del bosque dedicada al simulador *Forest Fire* vamos a tener únicamente cuatro elementos: árboles, fuego, agua y tierra. En esa zona no pueden existir celdas vacías. Cada celda tiene ocho vecinos (vecindario de Moore).

- Si un árbol débil queda cerca al fuego pierde el 25 % de su energía y si queda con menos de 10 % de energía se prende, convirtiéndose en fuego.
- Si un árbol queda cerca a una fuente de agua recupera 100 % de su energía. El agua que lo alimentó se agota, convirtiéndose en tierra.
- Si un árbol queda rodeado sólo por tierra genera un nuevo árbol en una de las celdas vecinas. Si el fuego queda cerca al agua, se extingue; y dura cinco ciclos, si permanece rodeado únicamente por tierra. Al extinguirse se convierte en tierra.
- Si el agua está rodeada por tierra, intenta moverse hacia el sur. El espacio que deja se convierte en tierra.
- La tierra puede generar fuego con una probabilidad de 10 % y agua con probabilidad de 5 %. Estos elementos en cada momento deciden la acción a realizar, la primera posible siguiendo el orden en que se presentan las reglas, y la realizan inmediatamente.

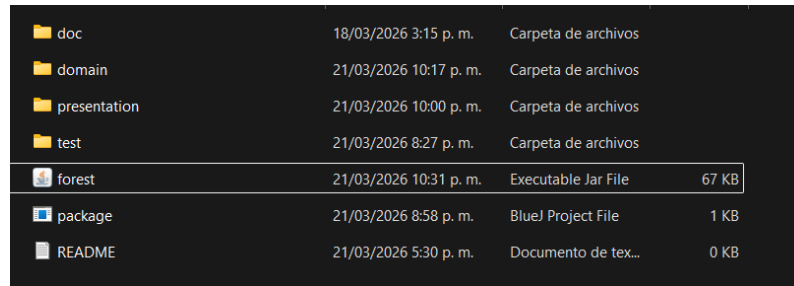
PARTE IV. Empaquetando el proyecto [En lab03.doc forest.jar]

En un **JAR** (Java Archive) se pueden empaquetar múltiples clases y archivos para distribuir y ejecutar artefactos software en un solo archivo comprimido.

1. Revise las opciones de BlueJ para empaquetar su programa en un archivo **.jar**. Genere el archivo correspondiente.

(a) Incluya una pantalla con evidencia de la existencia del archivo.

R/:



doc	18/03/2026 3:15 p. m.	Carpeta de archivos	
domain	21/03/2026 10:17 p. m.	Carpeta de archivos	
presentation	21/03/2026 10:00 p. m.	Carpeta de archivos	
test	21/03/2026 8:27 p. m.	Carpeta de archivos	
forest	21/03/2026 10:31 p. m.	Executable Jar File	67 KB
package	21/03/2026 8:58 p. m.	BlueJ Project File	1 KB
README	21/03/2026 5:30 p. m.	Documento de tex...	0 KB

2. Consulte el comando java para ejecutar un archivo **.jar**. ejecútenlo

(a) ¿qué pasa?

R/: El comando para ejecutar el archivo es **java -jar forest.jar**, lo que sucede al ejecutarlo es similar a cuando se compila ForestGUI en BlueJ, simplemente que se realiza desde la terminal de acuerdo con el Sistema Operativo que se use.

3. (a) ¿Qué ventajas tiene esta forma de entregar los proyectos? Explique claramente.

R/: Las ventajas de entregar proyectos de esta forma son las siguientes:

- **Mejora en la memoria:** dado que los objetos consumen mucha memoria RAM, al ejecutar el proyecto desde el CMD, se obtiene una eficiencia mucho más grande.
- **Ventajas cuando el programa falla:** Si el programa llega a fallar, el CMD puede arrojar una excepción (como NullPointerException) con el fin de saber que el programa falla.
- **Funcionable sin necesidad de BlueJ:** Cualquier persona que tenga por lo menos el archivo .jar pero no tenga BlueJ, puede ejecutar el proyecto desde la consola con el fin de confirmar que el proyecto es funcional.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

R/:

- **Juan Pablo Cuervo Contreras:** 32 horas.
- **David Felipe Ortiz Salcedo:** 32 horas.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

R/: El estado actual del laboratorio es culminado, aunque faltaron cosas por mejorar, dimos nuestro mejor esfuerzo para poder entender y aplicar conceptos aprendidos.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

R/: La práctica más útil en este laboratorio fue **Coding: All production code is pair programmed**, dado que el laboratorio al ser demasiado extenso, el hecho de dividirnos el trabajo nos benefició mucho para poder culminarlo.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/: Consideramos que el mayor logro fue haber podido intentar plasmar las distintas cosas que se pedían en el simulador como lo fueron las ardillas o el agua.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/: Consideramos que el mayor problema técnico es que a veces BlueJ molesta a ratos, es decir, a veces no deja compilar los casos de prueba.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R/: Comunicarnos todo el tiempo para saber qué faltaba y cómo poder plantear una solución. Nos comprometemos a mejorar el manejo del tiempo, ya que no tuvimos tanto tiempo como para poder pulir detalles restantes.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R/:

- **W3Schools. W3Schools — Tutoriales y referencias de programación.**

<https://www.w3schools.com/>

Fue la referencia más útil para poder entender ciertos conceptos.

